TypeRacer
打字練習遊戲

S1154005李育林
S1154002陳品璁

目錄

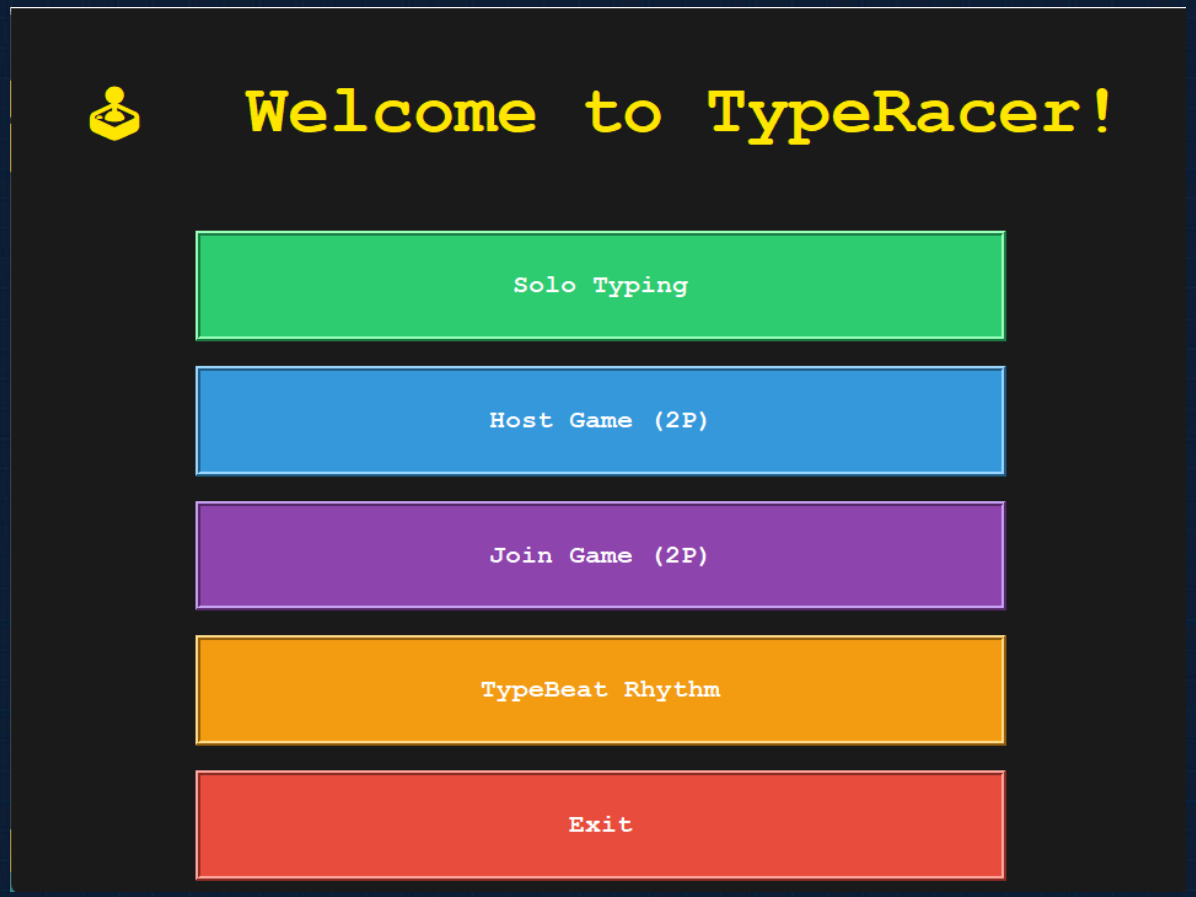
設計理念與動機


class Node:
def __init__(self, val=0, left=None
self.val = val
self.left = left
self.right = right
def inorder(node):
if node is None:
return
inorder(node.left)
print(node.val)
inorder(node.right)
def inorder_iterativuh(node):
# this is a iterative unctior
# sddf asdf sasddf
return
root = Node(12)
root.left = Node(6)
root.right = Node(4)


typeracer


monkeytype
wpm
76
acc
97%
Words per Minute
Errors

設計理念與動機



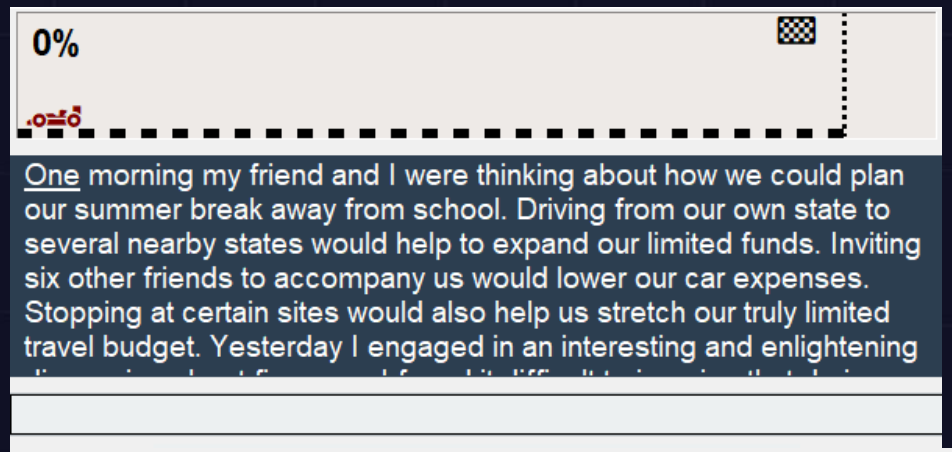
遊戲模式:

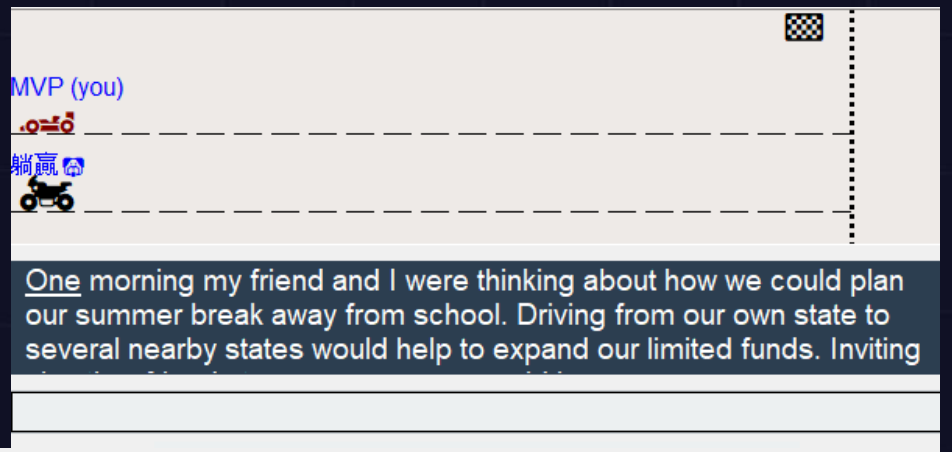
1. 單人打字練習
   - 經典文章打字練習

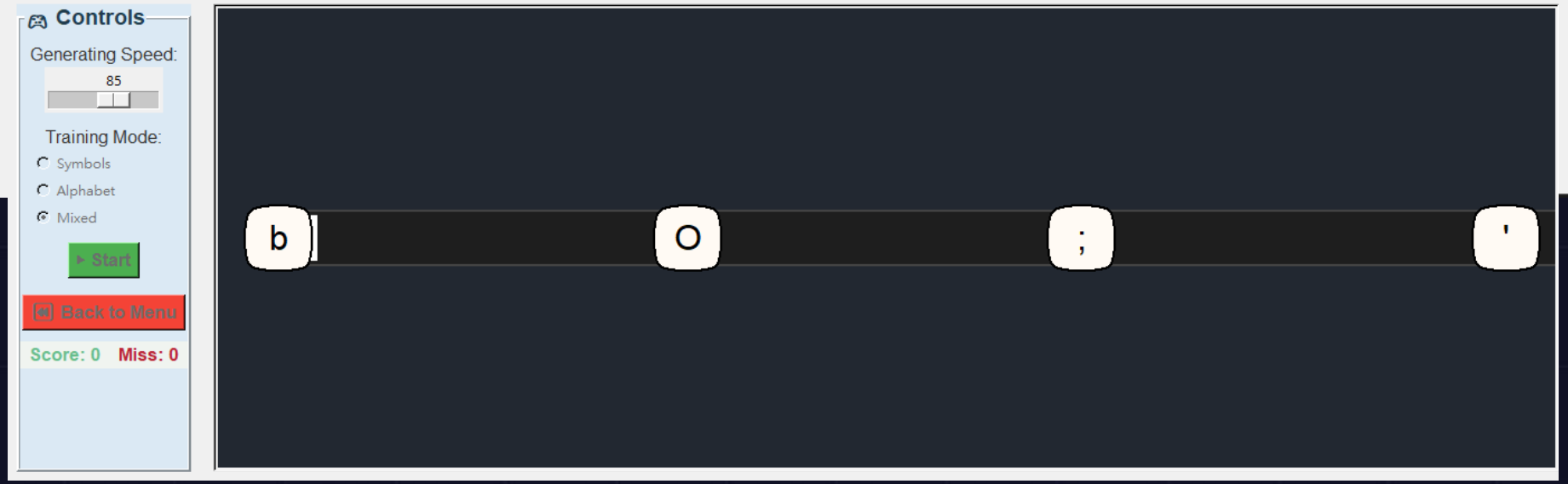
2. 雙人打字PK
   - host端與join端雙人連機比賽

3. 類音樂遊戲練習
   - 太鼓達人風格特殊打字訓練遊戲

設計理念與動機


0%
One morning my friend and I were thinking about how we could plan our summer break away from school. Driving from our own state to several nearby states would help to expand our limited funds. Inviting six other friends to accompany us would lower our car expenses. Stopping at certain sites would also help us stretch our truly limited travel budget. Yesterday I engaged in an interesting and enlightening


MVP (you)
躺贏
One morning my friend and I were thinking about how we could plan our summer break away from school. Driving from our own state to several nearby states would help to expand our limited funds. Inviting


Controls
Generating Speed:
85
Training Mode:
Symbols
Alphabet
Mixed
Start
Back to Menu
Score: 0
Miss: 0
b
O
;
'

實作執行方式

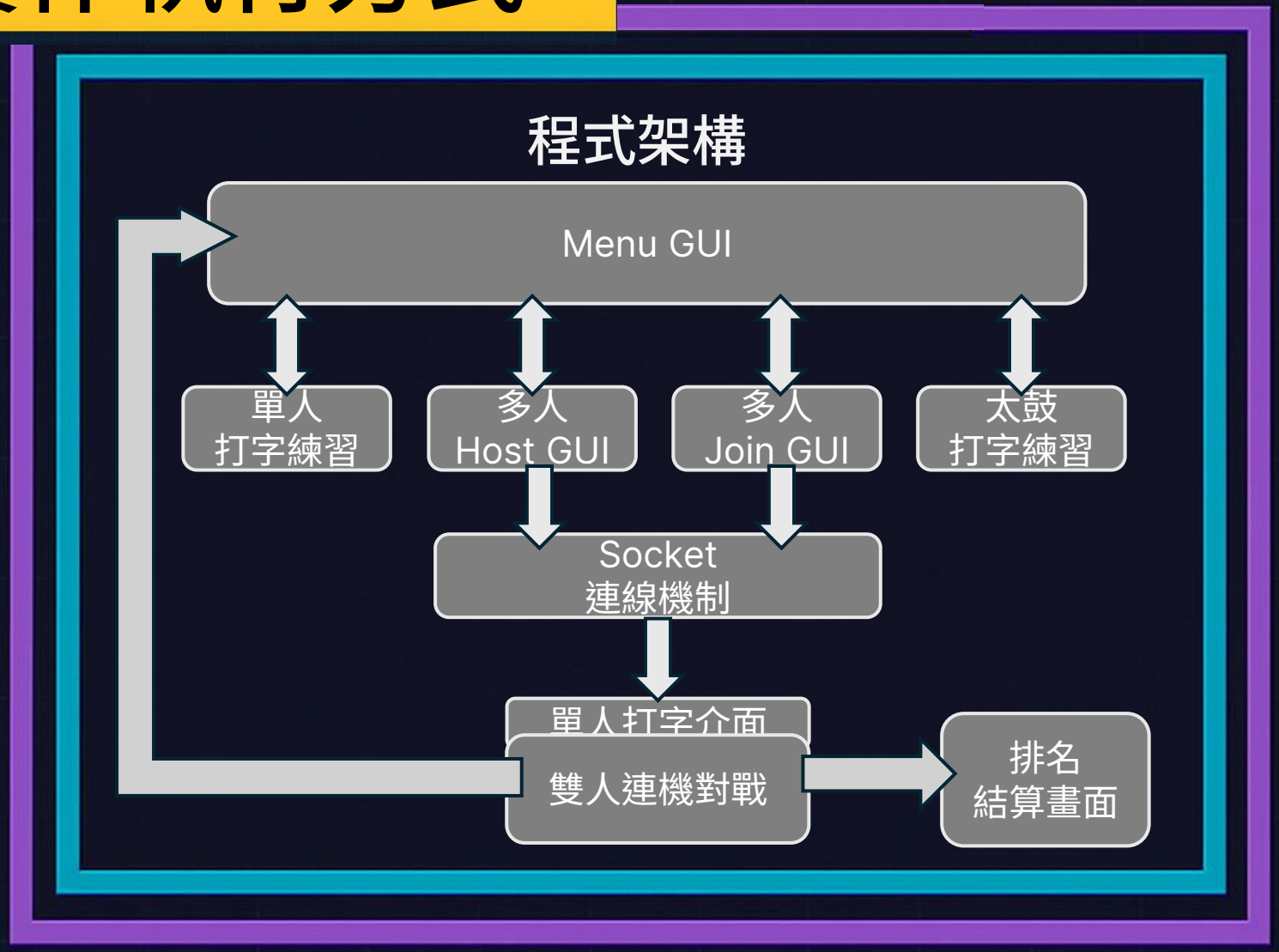
TypeRacer/

- main.py
- game.py
- host_gui.py
- join_gui.py
- multiplayer_game.py
- type_beat_trainer.py
- articles

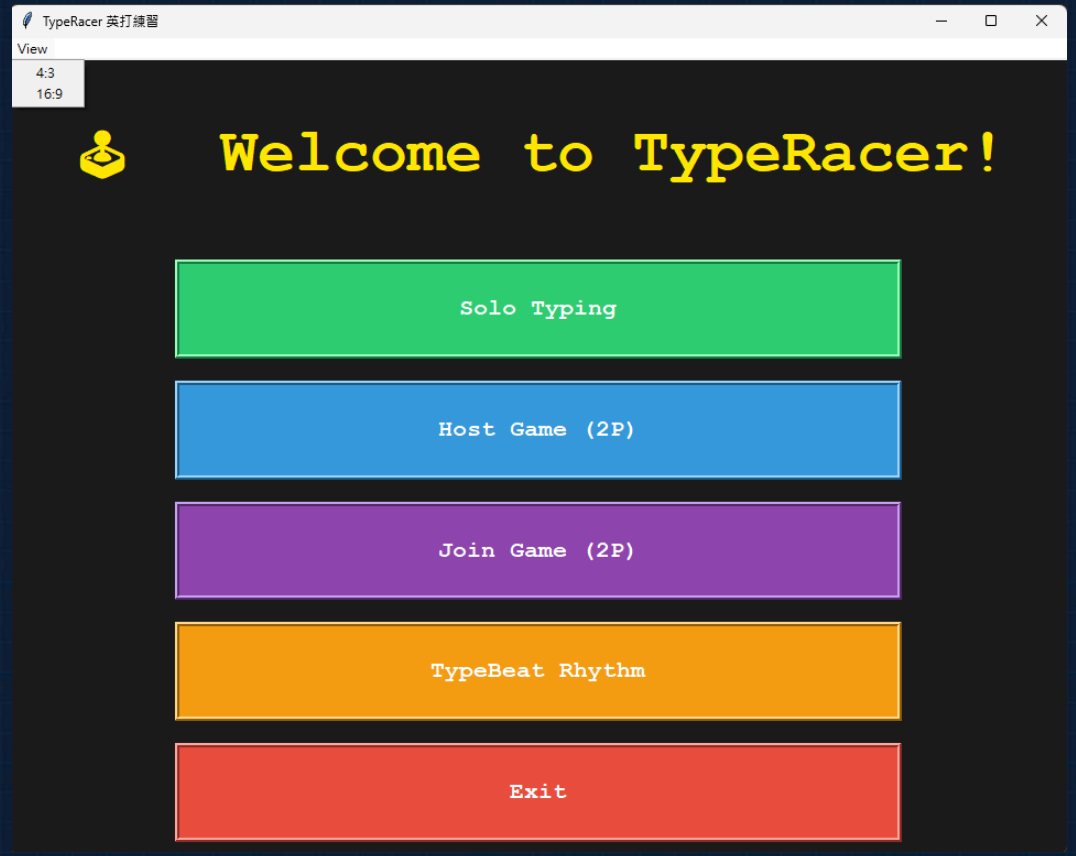
使用函式庫:

- **Time**
  - 打字用時計時
- **Random**
  - 隨機文本抽選&太鼓音符生成
- **Socket**
  - 建立連機遊戲連線
- **Threading**
  - 連機遊戲監聽

實作執行方式


程式架構
Menu GUI
單人
打字練習
多人
Host GUI
多人
Join GUI
太鼓
打字練習
Socket
連線機制
單人打字介面
雙人連機對戰
排名
結算畫面

實作執行-標題畫面



功能介紹:

- View menubar
  - 可依此快速調整視窗比例大小
- Menu GUI
  - 街機風格GUI介面
  - 管理每個模式的返回標題顯示
  - 選擇遊玩模式

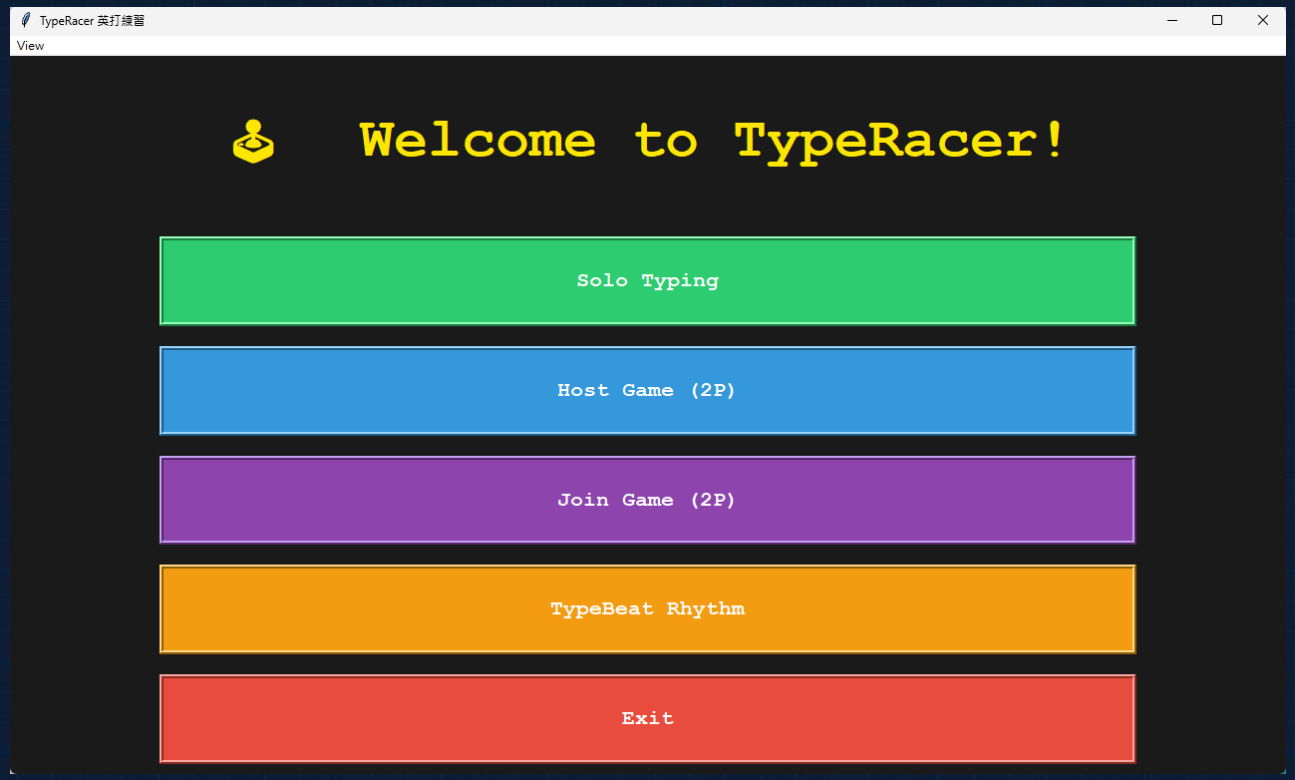
實作執行-Menu

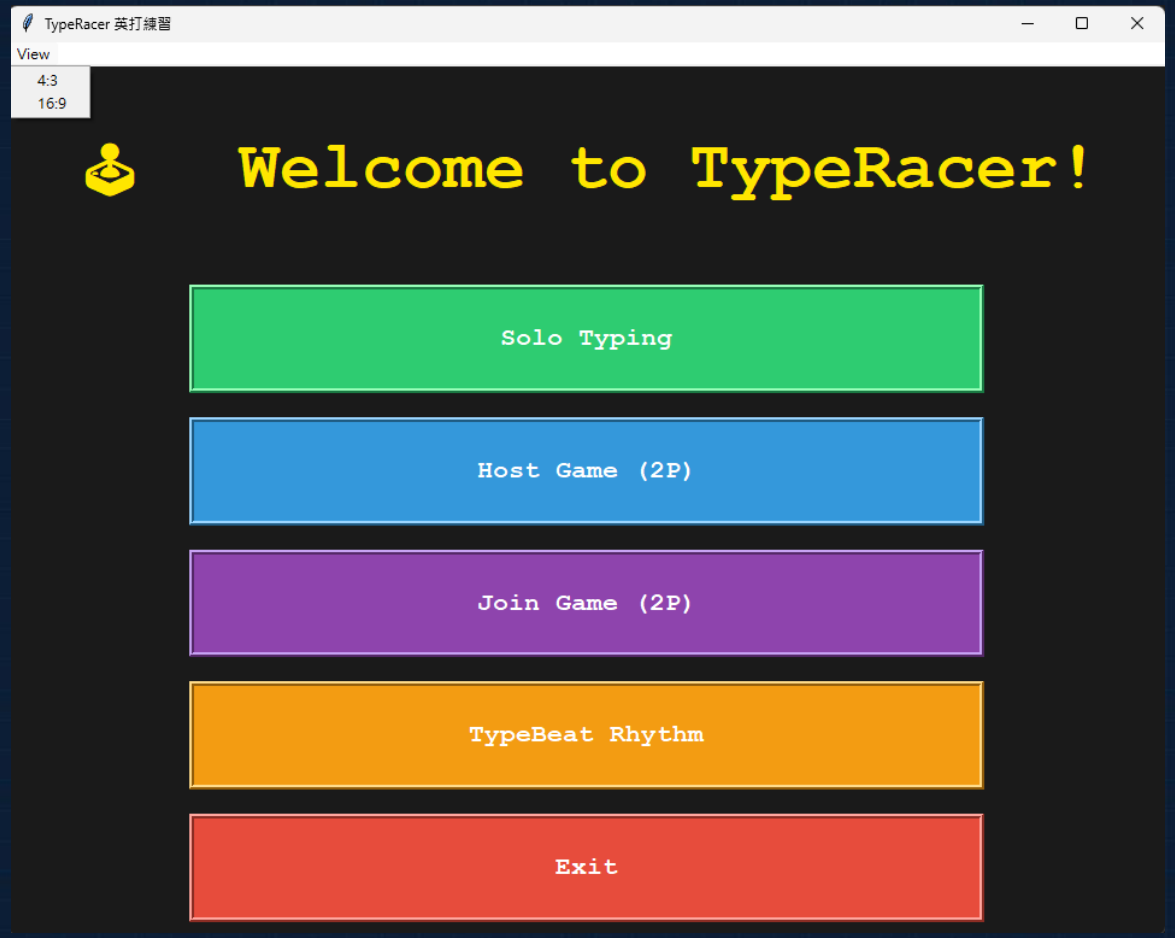
```python
def create_menu_bar(self):
    menubar = tk.Menu(self)
    view_menu = tk.Menu(menubar, tearoff=0)
    view_menu.add_command(label="4:3", command=lambda: self.set_ratio("4:3"))
    view_menu.add_command(label="16:9", command=lambda: self.set_ratio("16:9"))
    menubar.add_cascade(label= "View", menu=view_menu)
    self.config(menu=menubar)

def set_ratio(self, ratio):
    self.aspect_ratio = ratio
    self.set_resolution()

def set_resolution(self):
    screen_width = self.winfo_screenwidth()
    screen_height = self.winfo_screenheight()
    if self.aspect_ratio == "4:3":
        width = 960
        height = 720
    else:
        width = 1280
        height = 720
    x = (screen_width // 2) - (width // 2)
    y = (screen_height // 2) - (height // 2)
    self.geometry(f"{width}x{height}+{x}+{y}")
```

功能介紹:

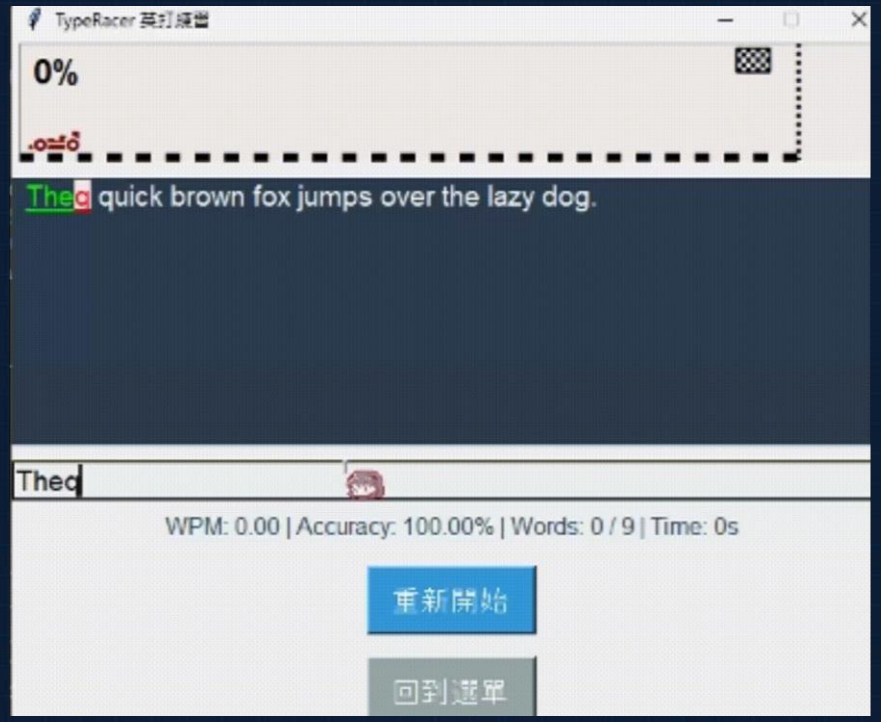
玩家可透過Menubar中的view menu選擇4:3或16:9的視窗大小，快速調整視窗，並且進入打字遊戲時也會跟著調整視窗比例大小

實作執行-Menu


TypeRacer 英打練習
View
Welcome to TypeRacer!
Solo Typing
Host Game (2P)
Join Game (2P)
TypeBeat Rhythm
Exit


TypeRacer 英打練習
View
4:3
16:9
Welcome to TypeRacer!
Solo Typing
Host Game (2P)
Join Game (2P)
TypeBeat Rhythm
Exit

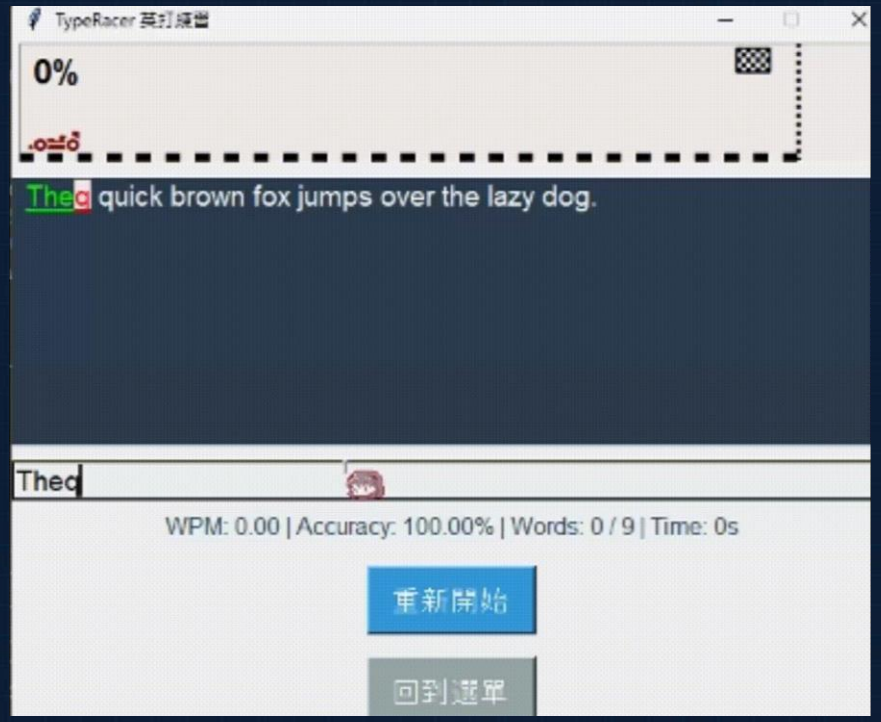
實作執行-單人遊戲



遊玩說明:

- 依照文章輸入單字進行比對
- 完成目前單字更新進度條&統計資訊
- 完成文章小車終點結算動畫

實作執行-單人遊戲



功能介紹:

- 賽車進度條
  - 顯示賽車風格打字進度條
- 文本框輸入錯誤比對
  - 底線顯示目前單字
  - 輸入正確/錯誤顯示
- 打字數據計算
  - WPM&準確率計算
  - 計時總打字時間
- 文章顯示自動換行
  - 文章過長隨進度自動調整顯示列

實作執行-單人遊戲

```
self.typing_entry.bind("<KeyRelease>", self.on_key_release)
```

```
def on_key_release(self, event):
    current_input = self.typing_entry.get()

    # 計算輸入錯誤次數
    if self.current_word_idx < len(self.target_words):
        target_word = self.target_words[self.current_word_idx]
        is_wrong = False
        # 判斷目前輸入與目前打到的文本單字是否有錯誤
        for j in range(min(len(current_input), len(target_word))):
            if current_input[j] != target_word[j]:
                is_wrong = True
                break
        if len(current_input) > len(target_word):
            is_wrong = True
        if is_wrong and not self.has_made_mistake:
            self.total_wrong_words += 1
            self.has_made_mistake = True

    self.render_article(current_iunput=current_input)
    if not self.is_typing:
        self.start_time = time.time()
        self.is_typing = True
        # 暫時關閉menu不能更改尺寸
        emptyMenu = tk.Menu()
        self.root.config(menu=emptyMenu)
```

文字輸入偵測

對輸入欄綁定按鍵輸入作爲偵測點，檢查打字輸入，並透過比對檢查回饋輸入正確/錯誤給玩家

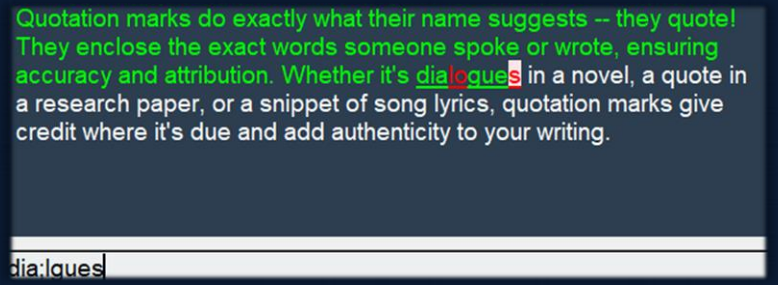
實作執行-單人遊戲

```
# 設定文本文字顏色樣式，用以區分打過的字，目前打到哪...
self.article_display.tag_config("correct", foreground="lime")
self.article_display.tag_config("incorrect", foreground="red")
self.article_display.tag_config("default", foreground=ARTICLE_FG, background=ARTICLE_BG)
self.article_display.tag_config("current_correct", foreground="lime", underline=1)
self.article_display.tag_config("current_incorrect", foreground="red", underline=1)
self.article_display.tag_config("current_pending", foreground=ARTICLE_FG, underline=1)
self.article_display.tag_config("extra_wrong", foreground="red", background="#ffe6e6", underline=1)
self.article_display.config(state="disabled")
```

```
            tag = "default"
            # 處理已經完成的單字文本顯示
            if i < len(self.typed_words):
                if j < len(self.typed_words[i]):
                    if self.typed_words[i][j] == char:
                        tag = "correct"
                    else:
                        tag = "incorrect"
                else:
                    tag = "incorrect"
            # 處理目前進行到的單字文本顯示
            elif i == self.current_word_idx:
                if j < len(current_iunput):
                    if current_iunput[j] == char:
                        tag = "current_correct"
                    else:
                        tag = "current_incorrect"
                else:
                    tag = "current_pending"
            self.article_display.insert(tk.END, char, tag)

        # 當目前單字多餘輸入時同樣在文本上顯示錯誤
        if i == self.current_word_idx and len(current_iunput) > len(word):
            extra_text = current_iunput[len(word):]
            for c in extra_text:
                self.article_display.insert(tk.END, c, "extra_wrong")
        self.article_display.insert(tk.END, " ", "default") # 不要忘了單字間的空格
    self.article_display.config(state="disabled")
    self.article_display.yview_scroll(self.line, "units")
```

文章文字比對

根據使用者輸入，實時比對文章文字，並提供錯誤提示，可以即時回饋輸入正確與錯誤



實作執行-單人遊戲

```
self.typing_entry.bind("<space>", self.on_space_press)
```

```
def on_space_press(self, event):
    typed_word = self.typing_entry.get().strip()
    if self.current_word_idx < len(self.target_words):
    # 若目前打的單字完全正確可以進入下一個單字
        if typed_word == self.target_words[self.current_word_idx]:
            self.typed_words.append(typed_word)
            self.current_word_idx += 1
            self.typing_entry.delete(0, tk.END)
            self.has_made_mistake = False
            self.render_article()
            self.auto_scroll_after_first_line_done()
            self.update_stats()
            self.update_progress_bar()

            # 當全部文章都打完時
            if self.current_word_idx >= len(self.target_words):
                self.typing_entry.config(state="disabled")
                self.update_stats(final=True)
                self.car_blink_at_finish(self.car)
                self.create_menu_bar()
    return "break"
```

單字輸入判斷

綁定空白鍵作爲判斷輸入正確的觸發點，檢查輸入是否和目前文章目標單字相同，並對文章顯示、進度條、數據進行更新

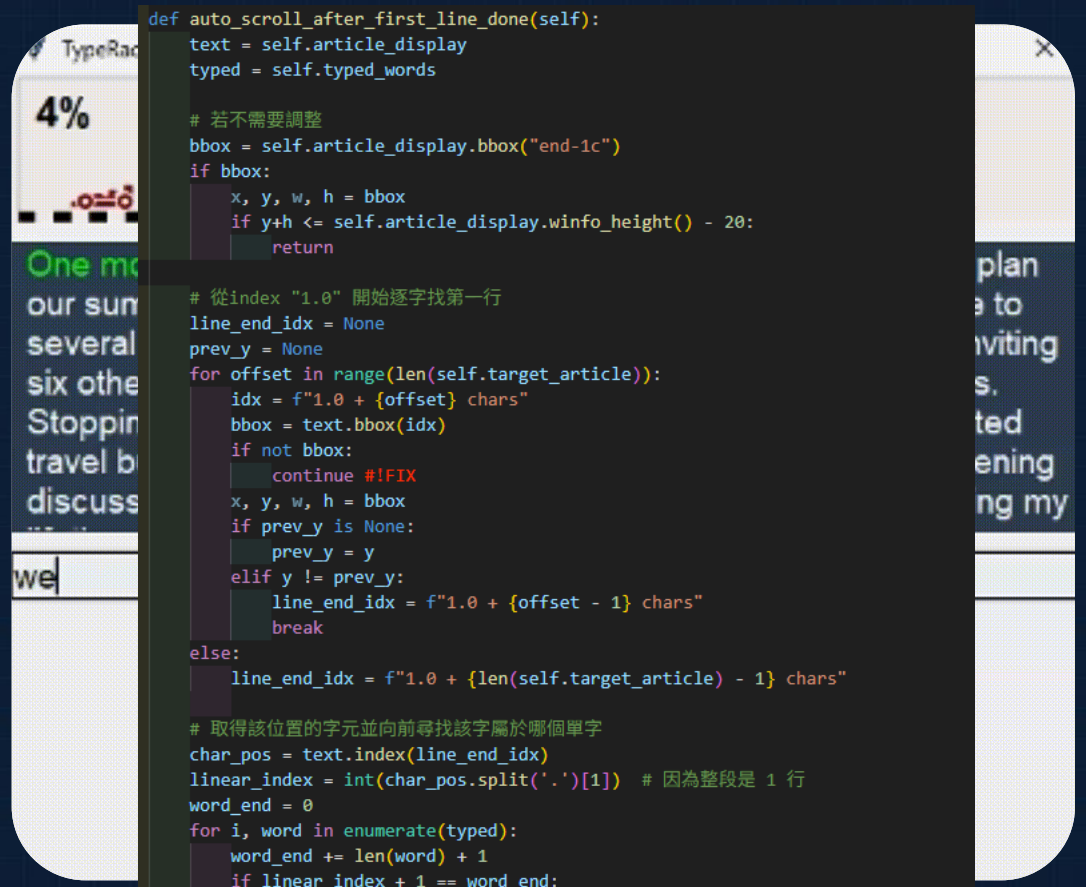
實作執行-單人遊戲

```
def update_progress_bar(self):
    if not hasattr(self, 'car'):
        return

    total_words = len(self.target_words)
    completed_words = len(self.typed_words)
    progress = completed_words / total_words if total_words > 0 else 0

    # 根據進度移動車子（總長度為 finish_line_x）
    target_x = int(progress * self.finish_line_x)
    progress_text = f"{int(progress * 100)}%"

    # 取得目前car位置並設定小車動畫參數
    car_current_coords = self.track_canvas.coords(self.car)
    car_current_x = car_current_coords[0] if car_current_coords else 0
    steps = 10
    dx = (target_x - car_current_x) /steps

    def animate(step):
        if step >= steps:
            self.track_canvas.coords(self.car, target_x, 30)
            self.track_canvas.itemconfig(self.progress_text, text=progress_text)
            return
        new_x = car_current_x + dx*step
        self.track_canvas.coords(self.car, new_x, 30)
        self.track_canvas.itemconfig(self.progress_text, text=progress_text)
        self.track_canvas.after(15, lambda: animate(step + 1))
    animate(0)
```

賽車進度條

當前面的單字輸入判定成功時，會來更新賽車進度，並依照完成比例來移動距離

實作執行-單人遊戲




```python
def auto_scroll_after_first_line_done(self):
    text = self.article_display
    typed = self.typed_words

    # 若不需要調整
    bbox = self.article_display.bbox("end-1c")
    if bbox:
        x, y, w, h = bbox
        if y+h <= self.article_display.winfo_height() - 20:
            return

    # 從index "1.0" 開始逐字找第一行
    line_end_idx = None
    prev_y = None
    for offset in range(len(self.target_article)):
        idx = f"1.0 + {offset} chars"
        bbox = text.bbox(idx)
        if not bbox:
            continue #!FIX
        x, y, w, h = bbox
        if prev_y is None:
            prev_y = y
        elif y != prev_y:
            line_end_idx = f"1.0 + {offset - 1} chars"
            break
    else:
        line_end_idx = f"1.0 + {len(self.target_article) - 1} chars"

    # 取得該位置的字元並向前尋找該字屬於哪個單字
    char_pos = text.index(line_end_idx)
    linear_index = int(char_pos.split('.')[1])  # 因為整段是 1 行
    word_end = 0
    for i, word in enumerate(typed):
        word_end += len(word) + 1
        if linear_index + 1 == word_end:
            self.line += 1
            break
```

文章換行自動捲動

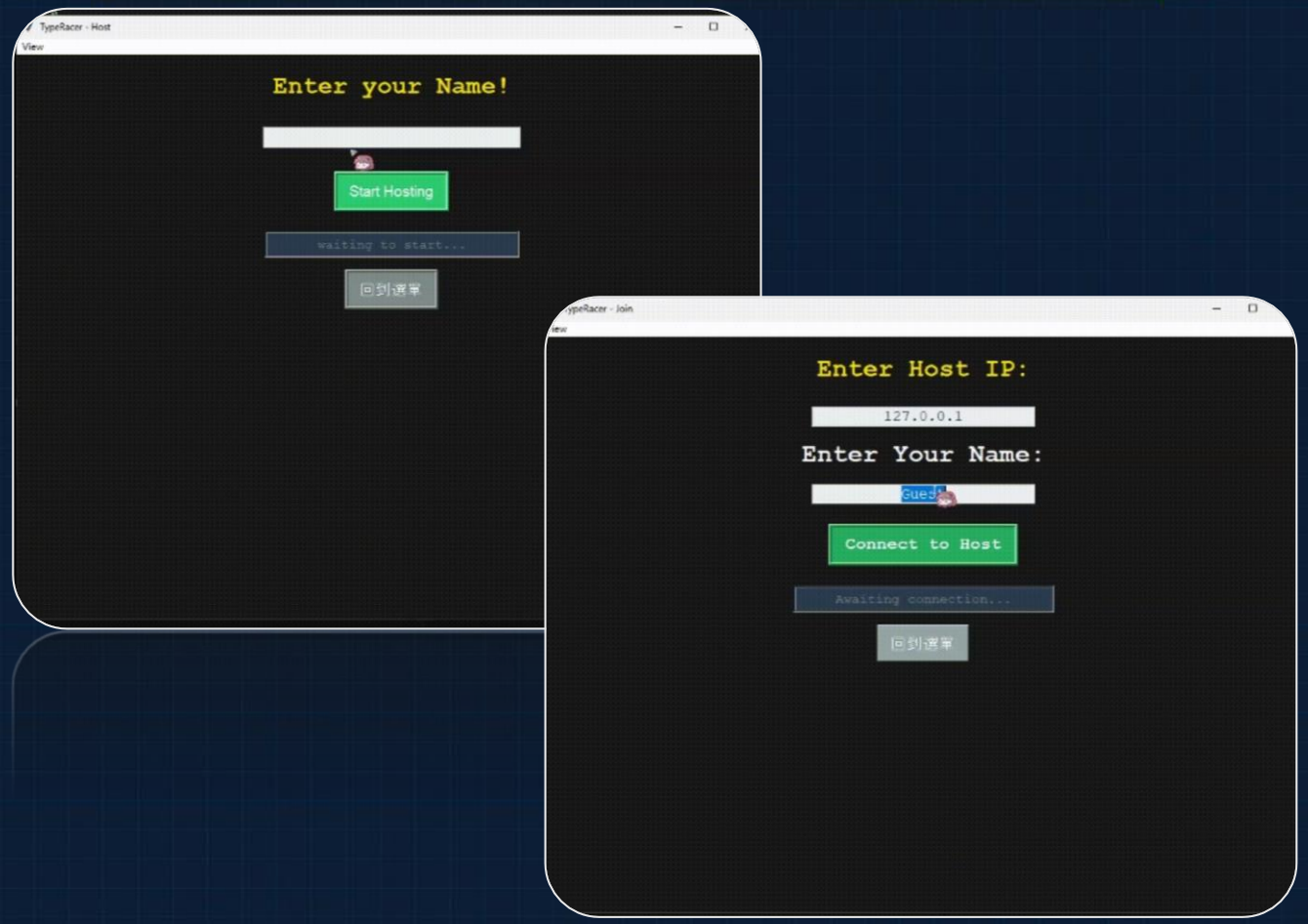
輸入到換行階段時，自動判斷剩下文章是否可以全部被顯示到畫面決定是否要向下捲動一行，讓玩家有更好的打字體驗!

實作執行-多人遊戲

Host端連線介面

Join端連線介面

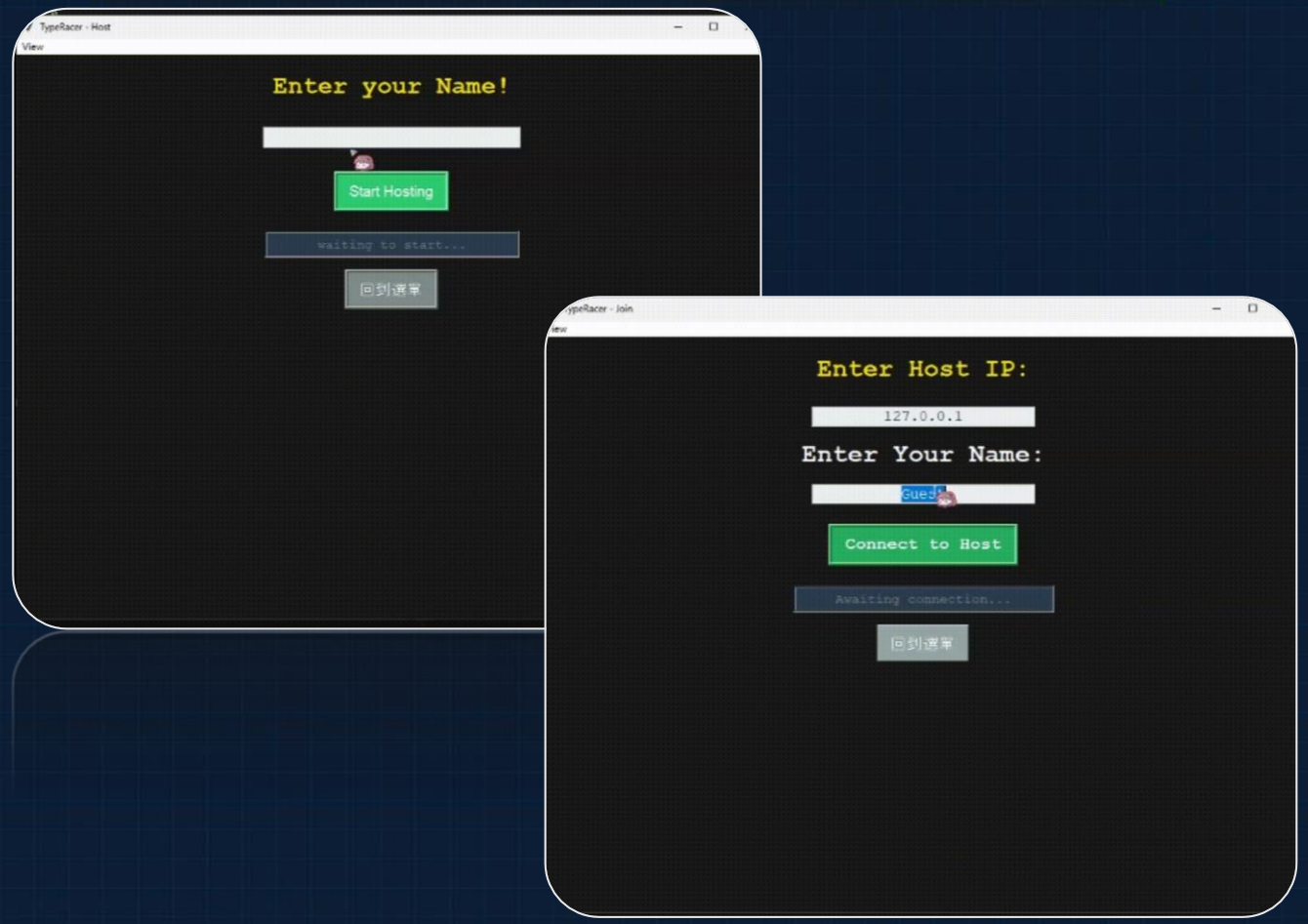
實作執行-多人遊戲



遊玩說明:

- 支援兩位玩家透過本地區網進行對戰
- 同時輸入相同文本，並透過車子動畫顯示各自的進度
- 最終結束顯示結算動畫發表名次

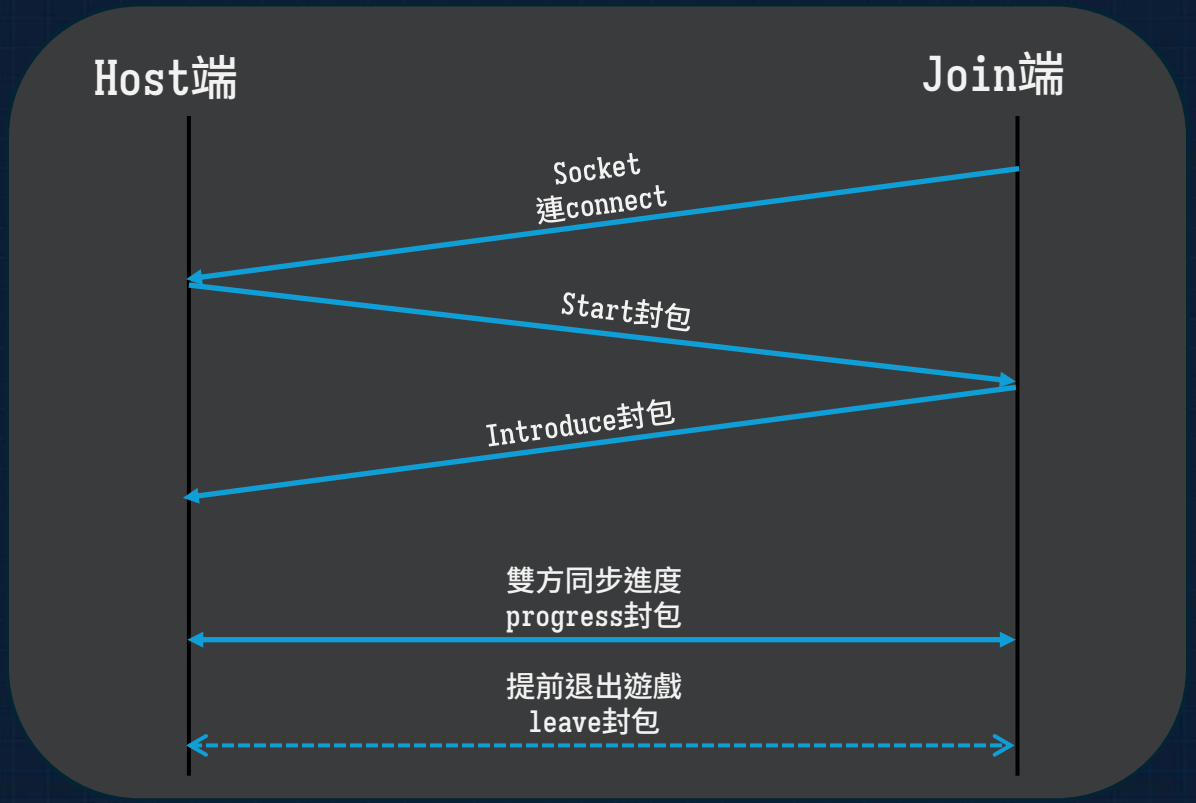
實作執行-多人遊戲



功能介紹:

- P2P socket連線機制
  - 設計introduce封包確保連線
  - threading同時雙向進度同步+GUI動畫
- 即時雙方進度更新
  - progress封包實時更新小車
- 中途退出
  - leave封包告知對手已經退出
- 結算畫面
  - 結束顯示結算畫面與動畫

實作執行-多人遊戲



P2P連線機制

透過Socket+TCP的連線，並透過自訂封包類型來達成整個連機遊戲的處理步驟與流程

多人遊戲-封包處理

```python
def handle_message(self, msg):
    with self.lock:
        if self.is_destroyed:
            return
        if msg["type"] == "progress":
            print("progress")
            player_id = msg["player_id"]
            name = msg["name"]
            self.players_progress[player_id] = msg["words"]
            self.add_or_update_car(player_id, name)
            # self.update_remote_car(player_id)

            if msg["finished"] and player_id not in self.finish_order:
                self.finish_order.append(player_id)
                if not self.game_over and len(self.finish_order) == len(self.players_progress):
                    self.game_over = True
                    self.show_results()

        elif msg["type"] == "introduce":
            print("introduce")
            name = msg["name"]
            player_id = msg["id"]
            self.add_or_update_car(player_id, name)
            if not self.ready_to_start:
                self.ready_to_start = True
                self.start_countdown()
```

```python
        elif msg["type"] == "start":
            print("start")
            name = msg["host_player_name"]
            self.target_article: str = msg["text"]
            self.target_words = self.target_article.split()
            self.render_article()
            self.update_stats()
            self.update_all_tracks()
            self.update_all_cars()
            if not self.ready_to_start:
                self.send_json({
                    "type": "introduce",
                    "name": self.player_name,
                    "id": self.player_id,
                })
                self.ready_to_start = True
                self.start_countdown()

        elif msg["type"] == "leave":
            print("leave")
            player_id = msg["id"]
            if player_id not in self.finish_order:
                self.finish_order.append(player_id)
                self.left_players.add(player_id)
                self.mark_player_as_left(player_id)
                self.check_game_over()
```

實作執行-多人遊戲

```python
def update_remote_car(self, player_id):
    if len(self.target_words) == 0 or player_id not in self.players_car_ids:
        return

    idx = list(self.players_car_ids.keys()).index(player_id)
    y_name = 50 + idx*50
    y_car = y_name + 10


    car = self.players_car_ids[player_id]
    name = self.players_name_ids[player_id]
    max_x = self.track_canvas.winfo_width() - 90
    progress_ratio = self.players_progress[player_id] / len(self.target_words) if len(self.target_words) >0 else 0

    # 位置計算
    car_current_coords = self.track_canvas.coords(car)
    car_current_x = car_current_coords[0]
    target_x = int(progress_ratio * max_x)
    steps = 10
    dx = (target_x - car_current_x) / steps

    def animate(step):
        if step >= steps:
            self.track_canvas.coords(car, target_x, y_car)
            self.track_canvas.coords(name, target_x ,y_name)
            return
        new_x = car_current_x + dx*step
        self.track_canvas.coords(car, new_x, y_car)
        self.track_canvas.coords(name, new_x, y_name)
        self.track_canvas.after(15, lambda: animate(step + 1))

    if player_id in self.players_name_ids and player_id in self.players_car_ids:
        animate(0)
```

即時雙方進度更新

透過Progress封包的溝通，達成雙機資料同步，透過進度條更新function更新畫面

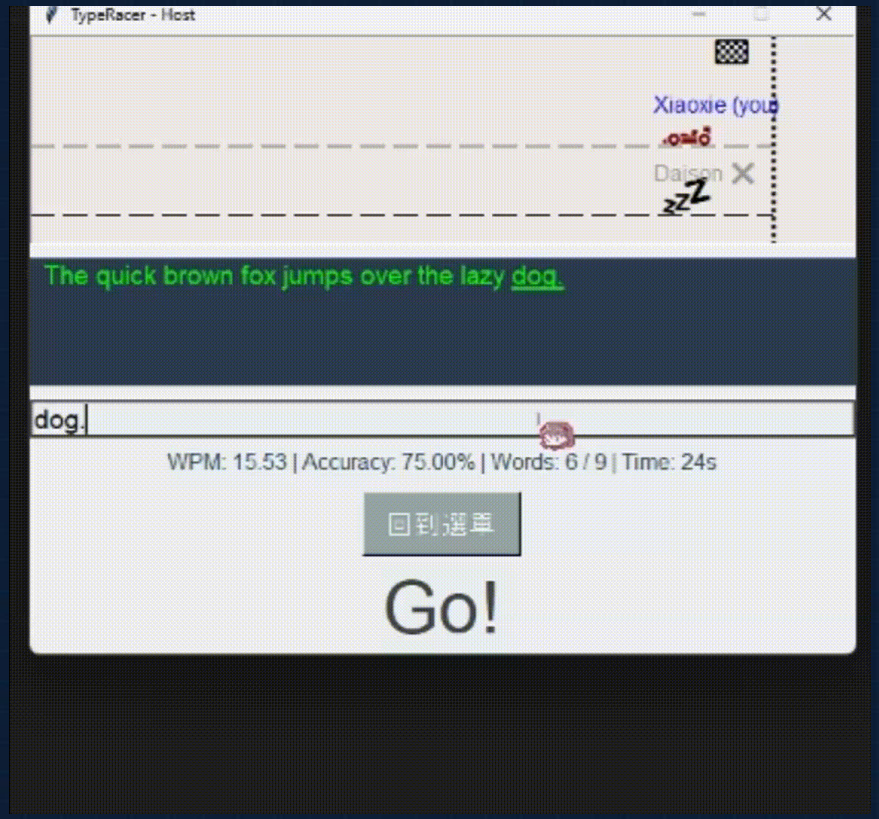
實作執行-多人遊戲


```python
def mark_player_as_left(self, player_id):
    # 標示名稱為灰色 + (Left)
    if player_id in self.players_name_ids:
        name = next(iter(self.left_players)).split("#")[0]
        self.track_canvas.itemconfig(self.players_name_ids[player_id], fill="#999999")
        self.track_canvas.itemconfig(self.players_name_ids[player_id], text=f"{name} ❌")

    # 將小車變灰或縮小
    if player_id in self.players_car_ids:
        self.track_canvas.itemconfig(self.players_car_ids[player_id], text="💤")
```

中途退出

當對手在遊戲進行到一半時返回標題或直接關閉遊戲都會發送leave封包，並更新小車狀態及最後結算畫面

實作執行-多人遊戲



結算畫面

雙方都完成打字或一方完成另一放中途退出時，產生結算畫面顯示排名以及結算動畫

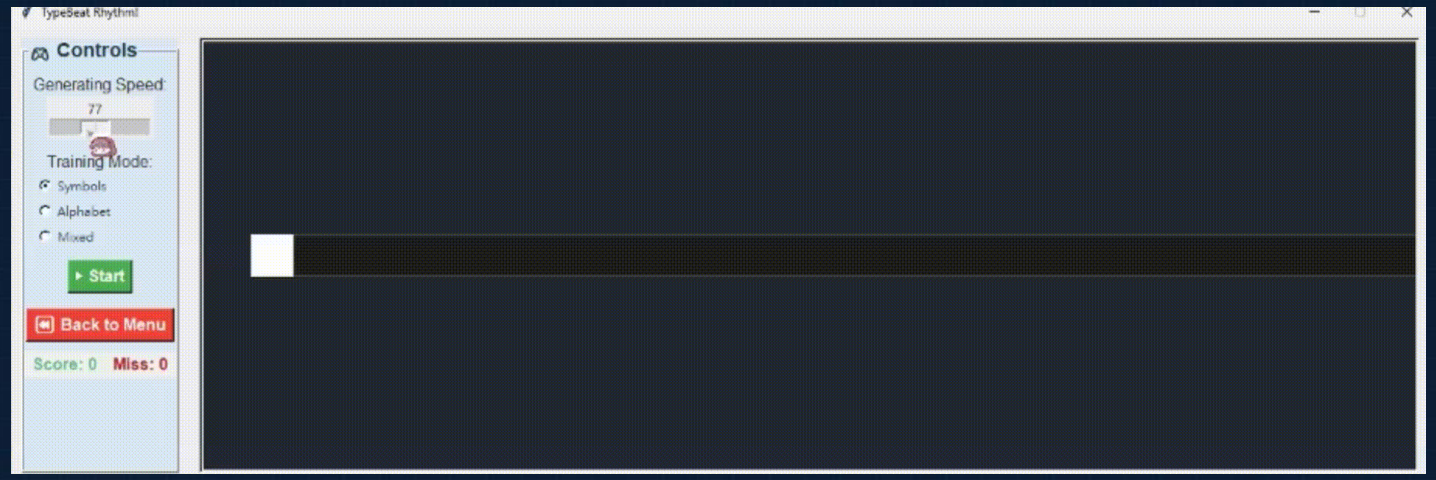
多人遊戲-結算畫面

```python
# 結算彩帶動畫
confetti = []
confetti_anim_id = [None] #保存彩帶canvas物件的id 後續停止使用
canvas = tk.Canvas(result_window, bg="#f8f9fa", highlightthickness=0)
canvas.place(relx=0, rely=0, relwidth=1, relheight=1)

def create_confetti():...

def animate_confetti():...

def stop_confetti():...

create_confetti()
animate_confetti()
result_window.after(5000, stop_confetti) # 5秒後停止彩帶掉落

# 冠軍閃爍動畫
def animate_winner(label):...
```

```python
# 冠軍閃爍動畫
def animate_winner(label):
    def blink(count=0):
        if count < 6:
            label.config(font=("Arial", 24, "bold"))
            label.config(font=("Arial", 22 + (2 if count % 2 == 0 else 0), "bold"))
            label.after(300, lambda: blink(count + 1))
        else:
            label.config(font=("Arial", 24, "bold"))
    blink()
```

```python
def create_confetti():
    canvas.update_idletasks()
    for _ in range(30):
        x = random.randint(0, max(50, 400))
        y = random.randint(-100, 0)
        color = random.choice(["#e74c3c", "#f1c40f", "#2ecc71", "#3498db", "#9b59b6"])
        shape = canvas.create_oval(x, y, x+6, y+6, fill=color, outline="")
        confetti.append(shape)

def animate_confetti():
    for shape in confetti:
        canvas.move(shape, 0, 5)
        coords = canvas.coords(shape)
        if coords[1] > 400:
            canvas.move(shape, 0, -500)
    confetti_anim_id[0] = canvas.after(50, animate_confetti)
```

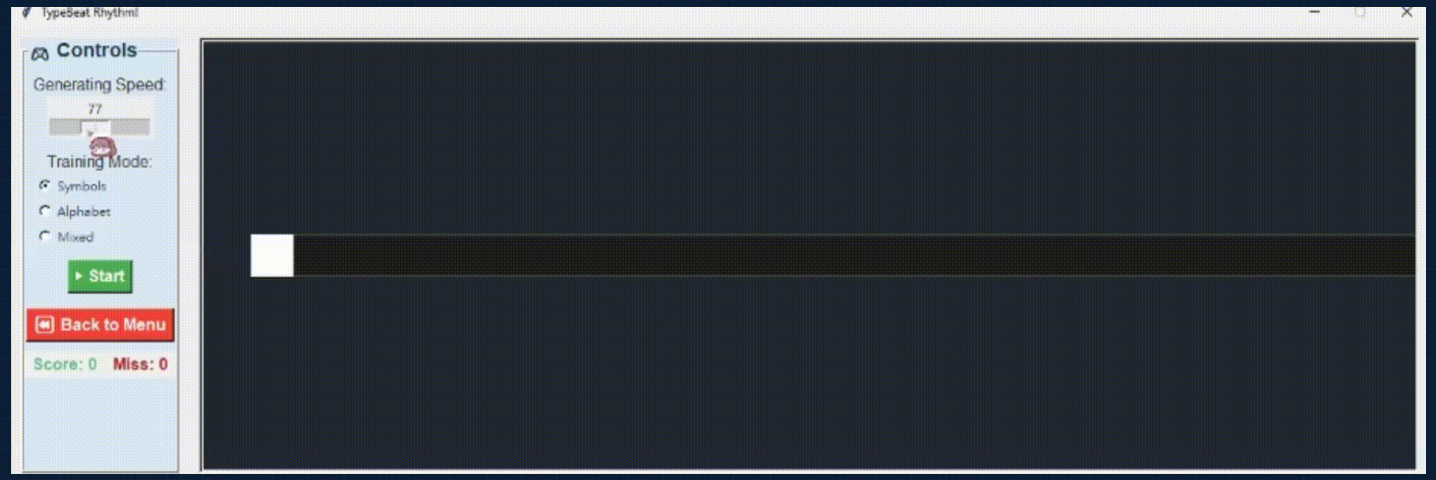
實作執行-太鼓模式



遊玩說明:

- 開始前可透過滑桿選擇音符(要打擊的文字)的生成速度
- 可以透過radioButton選擇要訓練的內容
- 看準文字，享受暢快的打擊體驗!

實作執行-太鼓模式



功能介紹:

- 打擊文字
  - 依照選定題目隨機生成要擊打文字
  - Move函數製作移動動畫
- 生成速度設定
  - 調整打擊文字生成密度
- 打擊回饋
  - Perfect/Miss回饋
  - 打擊閃爍特效
- 遊戲開始倒數/結束動畫

實作執行-太鼓模式

```python
def spawn_beat(self):
    if not self.running:
        return

    symbol = random.choice(self.current_set)
    w = self.canvas.winfo_width()
    h = self.canvas.winfo_height()
    y = h // 2

    # 畫出要輸入文字的圖形
    text_id = self.canvas.create_text(w + 30, y, text=symbol, font=("Arial", 24), fill="black", tags="beat", anchor="center")
    box_width = 60
    box_height = 60
    radius = 10
    x1 = w
    y1 = y - box_height // 2
    x2 = w + box_width
    y2 = y + box_height // 2
    rounded_rect = self.canvas.create_polygon(
        x1 + radius, y1,
```

```python
rounded_rect = self.canvas.create_polygon(
    x1 + radius, y1,
    x2 - radius, y1,
    x2, y1 + radius,
    x2, y2 - radius,
    x2 - radius, y2,
    x1 + radius, y2,
    x1, y2 - radius,
    x1, y1 + radius,
    smooth=True,
    outline="black", fill="#fffaf4", width=2, tag
text_box = rounded_rect
```

```python
    interval = int(60000 / (self.bpm*2))
    self.root.after(int(interval*3), self.spawn_beat)
```

/

```python
def move_beats(self):
    if not self.running:
        return

    for (box, item_id), _ in list(self.beats):
        if len(list(self.beats)) == 0:
            continue
        self.canvas.move(box, -4, 0)
        self.canvas.move(item_id, -4, 0)
        x, y = self.canvas.coords(item_id)
        # 處理若文字漏打超過判定線
        if x < self.judge_line - 10:
            self.canvas.delete(item_id)
            self.canvas.delete(box)
            self.beats = [b for b in self.beats if b[0][1] != item_id]
            self.show_feedback("Miss!", "#ff5555")
            self.miss_count += 1
            self.score_label.config(text=f"Score: {self.score}")
            self.miss_label.config(text=f"Miss: {self.miss_count}")
            if self.miss_count >= self.max_miss:
                self.end_game()
    self.root.after(10, self.move_beats)
```

打擊文字

透過random依題目隨機生成文字，再透過canvas畫出到畫面上，最後使用move函式製作文字移動動畫

實作執行-太鼓模式


```
# 計算每隔一段時間(bpm)就生成新的文字
interval = int(60000 / (self.bpm*2))
self.root.after(int(interval*3), self.spawn_beat)
```

生成速度設定

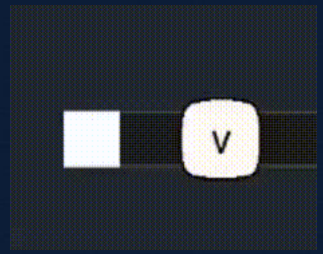
玩家可以透過左邊控制區的滑桿調整打擊方塊生成的速度

實作執行-太鼓模式

```python
# 命中或miss的回饋
def show_feedback(self, text, color):
    label = self.canvas.create_text(self.judge_line, self.canvas.winfo_height()//2 - 70,
                                    text=text, font=("Arial", 20, "bold"),
                                    fill=color, tags="feedback")
    self.canvas.after(500, lambda: self.canvas.delete(label))

def flash_hitbox(self, color="#00ff88"):
    h = self.canvas.winfo_height()
    box_height = 60
    box_width = 60
    x1 = (self.judge_line - box_width // 2) + 10
    y1 = h // 2 - box_height // 2
    x2 = (self.judge_line + box_width // 2) + 10
    y2 = h // 2 + box_height // 2

    flash_rect = self.canvas.create_rectangle(x1, y1, x2, y2, fill=color, width=0, tags="flash")
    self.canvas.tag_raise(flash_rect)
    self.root.after(100, lambda: self.canvas.delete(flash_rect))
```

打擊回饋

針對打擊輸入產生對應的Perfect和Miss回饋，增加打擊爽感



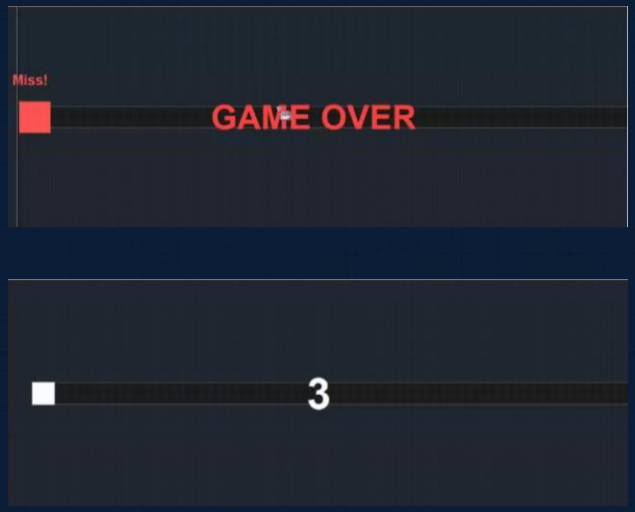
實作執行-太鼓模式

```
def end_game(self):
    self.running = False
    self.canvas.delete("beat")
    self.beats.clear()
    # 顯示 Game Over 動畫文字
    w = self.canvas.winfo_width()
    h = self.canvas.winfo_height()
    label = self.can
        w // 2, h //
        text="GAME O

    def flash(count=
        if count < 6
            current_
            self.can
            self.can
        else:
            self.can

    flash()
    self.score_label
    self.miss_label.
    self.enable_cont
```

```
def start_game(self, n):
    self.score = 0
    self.canvas.delete("gameover")
    self.canvas.delete("countdown")
    self.disable_controls()

    # 遊戲開始倒數
    if n == 0:
        self.running = True
        self.beats = []
        self.miss_count = 0
        self.score_label.config(text="Score: 0")
        self.miss_label.config(text="Miss: 0")
        self.canvas.delete("beat")
        self.current_set = self.symbol_sets[self.mode.get()]

        self.spawn_beat()
        self.move_beats()
    else:
        self.canvas.create_text(
                self.canvas.winfo_width()//2,
                self.canvas.winfo_height()//2,
                text=str(n), font=("Arial", 60, "bold"), fill="white", tags="countdown")
        self.canvas.after(1000, lambda: self.start_game(n-1))
```

遊戲開始倒數/結束動畫

遊戲開始結束撥放動畫

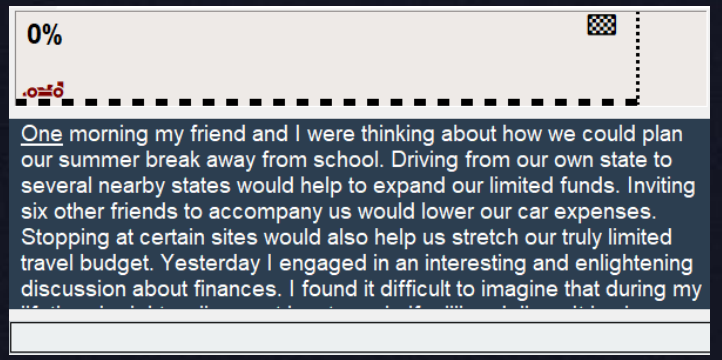


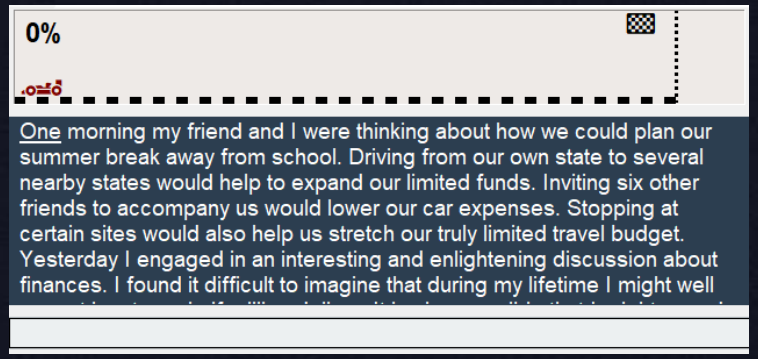
困難與解決方式

1. 讓玩家舒適的打字體驗

```
def set_resolution(self):
    screen_width = self.root.winfo_screenwidth()
    screen_height = self.root.winfo_screenheight()
    if self.aspect_ratio == "4:3":
        width = 480
        height = 360
    else:
        width = 640
        height = 360
    x = (screen_width // 2) - (width // 2)
    y = (screen_height // 4) - (height // 2)
    self.root.geometry(f"{width}x{height}+{x}+{y}")
```

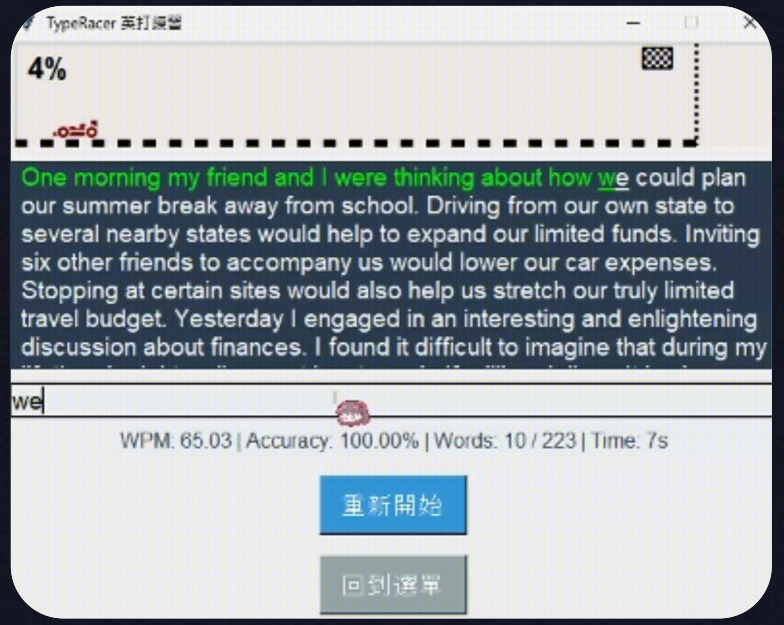
統一的視窗大小
限制文章顯示畫面





困難與解決方式

1. 讓玩家舒適的打字體驗

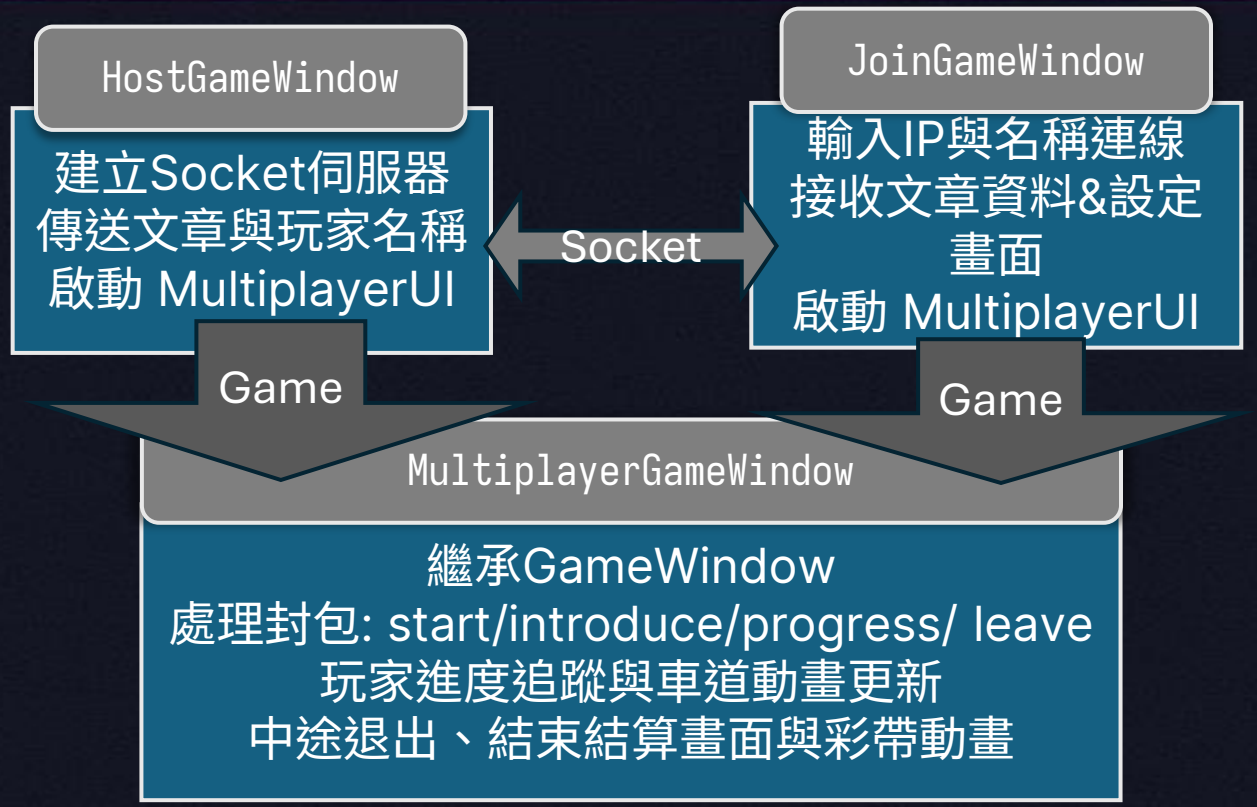


文章換行自動捲動

優秀的錯字回饋系統

困難與解決方式

2. 連機對戰連線設計與資料同步



結論&未來展望

結論

這份專題加入了Tkinter GUI+Socket連線，我在製作過程中遇到了不少問題也要去想不少額外的連線問題和使用者體驗，去設計出更好的介面和遊戲和連線機制。

展望與擴充方向

- 擴充雙人連機，支援更多人同時對戰
- 跨網域連線
- 排行榜系統

DEMO

Q&A